Neural Networks and Deep Learning (2026)

Assignment (2):

Tiny Shakespeare Memory Duel

Shiraz University

Due date: 1st of Khordad 23:59 [22/May (2026)]

1 Introduction

Predicting the next character in a sequence is a foundational task in recurrent neural networks. By learning the probability distribution of characters, a model can generate text that mimics the style, grammar, and structure of the source material. This task requires the model to maintain a temporal "memory" of preceding characters to predict the next one correctly, making it a perfect testbed for evaluating different recurrent architectures.

2 Problem Statement

In this homework, you will compare two classic recurrent neural network architectures: the Elman Network and the Jordan Network. Using the "Tiny Shakespeare" dataset, you will evaluate how different feedback mechanisms hidden-state feedback (Elman) versus output-distribution feedback (Jordan), influence the model's ability to capture linguistic patterns. You will examine how these architectures perform across training loss, character accuracy, and perplexity as they learn to generate coherent Shakespearean text. [**DATASET DOWNLOAD LINK**]

Since this dataset may be relatively large for quick experimentation, you are allowed to use only a portion of the text if needed. For example, you may train on 50%, 25%, or any other percentage of the original file, depending on your computational resources. The percentage used must be clearly reported in your final submission. The dataset should be treated as a character-level corpus. Each model will receive a sequence of characters and will learn to predict the next character at each time step. You must implement both the Elman and Jordan networks and compare their behavior under the same experimental settings.

In the text generation stage, you must also experiment with **temperature**. Temperature is a sampling parameter that controls how random the generated text is. A **low temperature** makes the model more confident and conservative, so it tends to choose high-probability characters and generate more predictable but sometimes repetitive text. A **high temperature** increases randomness, which can make the output more creative or diverse, but also more noisy and less coherent.

3 Tasks

1. After training both models with enough epochs, **generate text samples from both models** using the same seed text and compare the quality of the generated output. The **seed text** is a short initial sequence used to start text generation. The model uses it as context to predict subsequent characters, and using the same seed ensures a fair comparison between models.

2. **Report your results using different evaluation metrics** such as final character accuracy, and perplexity (PPL).
3. **Experiment with different temperatures** and discuss their effect on generation. Generated text samples at different temperatures (e.g., 0.4, 0.8, and 1.2) to highlight the trade-off between model stability and creative variety.
4. **Compare the two architectures** in terms of convergence speed, stability, validation performance, and generated text quality.

Notes:

- Allowed programming languages: Python, MATLAB
- Any sign of cheating would result in a zero grade for this assignment.
- You should upload your submissions at:
  https://quera.org/course/add_to_course/course/27529/
  Password: nn2026
  All of the files should be in a ZIP file named in this format:
  "FirstNameFamilyName-SudentNumber.zip"
  Ex: "AmirZamani-4053040.zip"
- Your reports should be in a PDF file including: key points of your implementation, explanation of your chosen approach, reports of your final results and answers of assignment questions (if given).
- If you do not have **Anaconda** or **Python** installed, you can download Anaconda from the following link: [**DOWNLOAD LINK**]
- Use the following **domestic PyPi mirror** to install the required libraries:
  https://package-mirror.liara.ir/repository/pypi/simple
  For more information about this mirror, visit: **https://liara.ir/mirrors/pypi/**
  Example: "`pip install minisom --index-url https://package-mirror.liara.ir/repository/pypi/simple`**"**
  If this mirror doesn't work, try Runflare instead: **https://runflare.com/mirrors/pypi-mirror/**
